## C# coding challenges and asp.net core concepts practice code for interviews:

## Check if the string is Palindrome:

```
public static bool IsPalindrome(string str)
{
    str = str.ToLower();  // ignore case
    int left = 0, right = str.Length - 1;

    while (left < right)
    {
        if (str[left] != str[right])
            return false;

        left++;
        right--;
    }

    return true;
}
```

A **palindrome string** is one that reads the same forward and backward.

### Example

- `"madam"` → ✅ Palindrome
- `"racecar"` → ✅ Palindrome
- `"hello"` → ❌ Not Palindrome

**Check if the given number is a prime number:**

```
public static bool IsPrime(int number)
{
    if (number <= 1)
        return false;

    for (int i = 2; i <= Math.Sqrt(number); i++)
    {
        if (number % i == 0)
            return false;
    }

    return true;
}
```

A **prime number** is a number greater than **1** that has no divisors other than **1** and itself.

### ✅ Examples of Prime Numbers

```
2, 3, 5, 7, 11, 13, 17, 19, 23, 29 ...
```

- `2` → prime (only divisible by 1 and 2)
- `3` → prime
- `4` → not prime (divisible by 2)
- `5` → prime
- `9` → not prime (divisible by 3)

**Count the frequencies of each character in a string:**

```
public static void FindCharFrequency(string str)
{
    Dictionary<char, int> freq = new Dictionary<char, int>();

    foreach (char ch in str)
    {
        if (ch == ' ') continue; // skip spaces

        if (freq.ContainsKey(ch))
            freq[ch]++;
        else
            freq[ch] = 1;
    }

    foreach (var item in freq)
    {
        Console.WriteLine($"{item.Key} → {item.Value}");
    }
}
```

### Example

Input:

```
arduino

"programming"
```

Frequencies:

- `p → 1`
- `r → 2`
- `o → 1`
- `g → 2`
- `a → 1`
- `m → 2`
- `i → 1`
- `n → 1`

**Find the factorial of a number:**

```
public static long Factorial(int n)
{
    long result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}
```

The **factorial** of a number n (written as n!) is the product of all positive integers from 1 to n.

## ✅ Examples

- `0! = 1` (by definition)
- `1! = 1`
- `3! = 3 × 2 × 1 = 6`
- `5! = 5 × 4 × 3 × 2 × 1 = 120`
- `7! = 7 × 6 × 5 × 4 × 3 × 2 × 1 = 5040`

### Formula:

```
n! = n × (n-1) × (n-2) × ... × 2 × 1
```

**Find the first non-repeating character in a string:**

```
public static char FirstNonRepeatingChar(string str)
{
    Dictionary<char, int> freq = new Dictionary<char, int>();

    // Count frequencies
    foreach (char ch in str)
    {
        if (freq.ContainsKey(ch))
            freq[ch]++;
        else
            freq[ch] = 1;
    }

    // Find first char with frequency = 1
    foreach (char ch in str)
    {
        if (freq[ch] == 1)
            return ch;
    }

    return '\0'; // if none found
}
```

Copy code

### Example 1

Input:

```
arduino

"swiss"
```

- `s → appears 3 times`
- `w → appears 1 time` ✅ (first unique)
- `i → appears 1 time` but comes later

Output:

```
nginx

w
```

**Reverse a string:**

```
public static string ReverseString(string str)
{
    char[] chars = str.ToCharArray();
    string result = "";

    for (int i = chars.Length - 1; i >= 0; i--)
    {
        result += chars[i]; // appending characters from end
    }

    return result;
}
```

### Example

Input:

```
"programming"
```

Output:

```
"gnimmargorp"
```

**Reverse a string using StringBuilder class:**

```
public static string ReverseString(string str)
{
    StringBuilder sb = new StringBuilder();

    for (int i = str.Length - 1; i >= 0; i--)
    {
        sb.Append(str[i]);
    }

    return sb.ToString();
}
```

**Reverse an integer array:**

```
public static void ReverseArray(int[] arr)
{
    int left = 0;
    int right = arr.Length - 1;

    while (left < right)
    {
        // Swap elements
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;

        left++;
        right--;
    }
}
```

### Example

Input:

```
[10, 20, 30, 40]
```

Output:

```
[40, 30, 20, 10]
```

**Find the second largest number in an integer array:**

```
public static int FindSecondLargest(int[] arr)
{
    if (arr.Length < 2)
        return -1; // not enough elements

    int first = int.MinValue;
    int second = int.MinValue;

    foreach (int num in arr)
    {
        if (num > first)
        {
            second = first;
            first = num;
        }
        else if (num > second && num < first)
        {
            second = num;
        }
    }

    return second == int.MinValue ? -1 : second; // -1 means no second largest
}
```

Copy code

## ✅ Examples

### Example

Input:

```
[5, 1, 9, 3, 7]
```

- Largest = 9
- Second largest = 7

Output:

```
7
```

## Count Vowels and Consonants in a string:

Copy code

```
public static void CountVowelsAndConsonants(string str)
{
    int vowelsCount = 0, consonantsCount = 0;
    string vowels = "aeiou";

    foreach (char ch in str.ToLower())
    {
        if (char.IsLetter(ch)) // process only alphabets
        {
            if (vowels.Contains(ch))
                vowelsCount++;
            else
                consonantsCount++;
        }
    }

    Console.WriteLine($"Vowels count = {vowelsCount}, Consonants count = {consonantsCount}");
}
```

### Example

Input:

```
"Programming"
```

- Vowels: `o, a, i` → 3
- Consonants: `P, r, g, r, m, m, n, g` → 8

Output:

```
Vowels: 3, Consonants: 8
```

**Print Fibonacci series up to n number:**

```
public static void PrintFibonacci(int n)
{
    int first = 0, second = 1;

    for (int i = 1; i <= n; i++)
    {
        Console.Write(first + " ");
        int next = first + second;
        first = second;
        second = next;
    }
}
```

The **Fibonacci series** starts with 0 and 1. Every next number is the sum of the previous two numbers:

```
0, 1, 1, 2, 3, 5, 8, 13, ...
```

### Example

Input: `n = 8`

Output:

```
0, 1, 1, 2, 3, 5, 8, 13
```

**Bubble sort an integer array:**

```
public static void BubbleSort(int[] arr)
{
    int n = arr.Length;

    for (int i = 0; i < n - 1; i++)
    {
        bool swapped = false;

        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                // swap arr[j] and arr[j+1]
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;

                swapped = true;
            }
        }

        // Optimization: if no swaps happened, array is already sorted
        if (!swapped)
            break;
    }
}
```

### Example

Input:

```
[5, 2, 9, 1, 5]
```

Steps:

1. Compare 5 & 2 → swap → `[2, 5, 9, 1, 5]`
2. Compare 5 & 9 → no swap
3. Compare 9 & 1 → swap → `[2, 5, 1, 9, 5]`
4. Compare 9 & 5 → swap → `[2, 5, 1, 5, 9]`

   ... repeat until fully sorted

Output:

```
[1, 2, 5, 5, 9]
```

**Find duplicates numbers in an integer array using Dictionary:**

```csharp
public static void FindDuplicates(int[] arr)
{
    Dictionary<int, int> freq = new Dictionary<int, int>();

    foreach (int num in arr)
    {
        if (freq.ContainsKey(num))
            freq[num]++;
        else
            freq[num] = 1;
    }

    Console.WriteLine("Duplicate elements:");
    foreach (var item in freq)
    {
        if (item.Value > 1)
            Console.WriteLine($"{item.Key} → {item.Value} times");
    }
}
```

### Example

Input:

```
csharp

[1, 2, 3, 2, 4, 5, 1]
```

Output:

```


1, 2
```

**Find duplicates numbers in an integer array using HashSet:**

```
public static void FindDuplicates(int[] arr)
{
    HashSet<int> seen = new HashSet<int>();
    HashSet<int> duplicates = new HashSet<int>();

    foreach (int num in arr)
    {
        if (!seen.Add(num)) // Add returns false if num already exists
        {
            duplicates.Add(num);
        }
    }

    if (duplicates.Count > 0)
    {
        Console.WriteLine("Duplicate elements: " + string.Join(", ", duplicates));
    }
    else
    {
        Console.WriteLine("No duplicates found.");
    }
}
```

### Example

Input:

```
csharp

[1, 2, 3, 2, 4, 5, 1]
```

Output:

```


1, 2
```

**Swap two numbers without using temp variable:**

```
public static void Swap(ref int x, ref int y)
{
    x = x + y;  // Step 1
    y = x - y;  // Step 2
    x = x - y;  // Step 3
}
```

### Example 1

Input:

```
ini

a = 5, b = 10
```

After swap:

```
ini

a = 10, b = 5
```

**Swap two numbers without using temp variable using bitwise method:**

### ✅ Using XOR (Bitwise) Method

```
csharp

public static void SwapXOR(ref int x, ref int y)
{
    x = x ^ y;
    y = x ^ y;
    x = x ^ y;
}
```

**Search a number in an integer array using linear search:**

Linear Search (also called sequential search) is a search algorithm where we check each element of a list/array one by one until we find the target element or reach the end of the list.

- It works on both sorted and unsorted arrays.
- Simple but not very efficient for large datasets.

```csharp
public static int LinearSearch(int[] arr, int target)
{
    for (int i = 0; i < arr.Length; i++)
    {
        if (arr[i] == target)
            return i; // return index
    }
    return -1; // not found
}
```

### Example 1

Input:

```csharp
arr = [10, 20, 30, 40], x = 30
```

Output:

```pgsql
Element 30 found at index 2
```

## Search a number in an integer array using binary search:

**Binary Search** is an **efficient search algorithm** used to find the **position of a target element** in a **sorted array**.

- Unlike linear search, it **does not check every element**.
- Instead, it **repeatedly divides the search interval in half**

### How it Works:

1. Start with the entire **sorted array**.
2. Find the **middle element**.
3. Compare the middle element with the **target**:
   - If equal → target found.
   - If target < middle → search in the **left half**.
   - If target > middle → search in the **right half**.
4. Repeat steps 2–3 until the target is found or the search space is empty.

```
public static int BinarySearch(int[] arr, int target)
{
    int left = 0;
    int right = arr.Length - 1;

    while (left <= right)
    {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target)
            return mid;
        else if (arr[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }

    return -1; // not found
}
```

## 🔎 Example 1

**Array:**

```csharp
[1, 3, 5, 7, 9, 11]
```

**Target:**

```
7
```

**Steps:**

1. Middle element = 5 → 7 > 5 → search right half `[7, 9, 11]`
2. Middle element = 9 → 7 < 9 → search left half `[7]`
3. Middle element = 7 → ✅ found at index `3`

**Output:**

```pgsql
Element 7 found at index 3
```

## Binary to Decimal conversion:

### 🔎 Problem:

Convert a **binary number** (base 2) to **decimal number** (base 10).

**Binary number:** uses only `0` and `1`

**Decimal number:** uses `0–9`

### 🔢 Logic:

1. Each binary digit has a **position value** starting from **right to left** (0-based).
2. Multiply each binary digit by `2^position`.
3. Sum all these values to get the decimal number.

**Formula:**

```arduino
Decimal = Σ (bit × 2^position)
```

### Example

Binary: `1101`

Step by step:

```arduino
Rightmost bit = 1 → 1 × 2^0 = 1
Next bit = 0 → 0 × 2^1 = 0
Next bit = 1 → 1 × 2^2 = 4
Next bit = 1 → 1 × 2^3 = 8
Sum = 8 + 4 + 0 + 1 = 13
```

Output:

```ini
Decimal = 13
```

```
public static int BinaryToDecimal(string binary)
{
    int decimalNumber = 0;
    int power = 1; // 2^0

    for (int i = binary.Length - 1; i >= 0; i--)
    {
        if (binary[i] == '1')
            decimalNumber += power;

        power *= 2;
    }

    return decimalNumber;
}
```

**Decimal to Binary conversion:**

### 🔎 Problem:

Convert a **decimal number** (base 10) to a **binary number** (base 2).

**Decimal number:** uses digits `0-9`
**Binary number:** uses only `0` and `1`

### 🔢 Logic:

1. Divide the decimal number by 2.
2. Write down the **remainder** (0 or 1).
3. Update the number = quotient of the division.
4. Repeat steps 1–3 until the number becomes 0.
5. **Binary number** = all remainders written in **reverse order** (from last remainder to first).

### Example

Decimal: `18`

Step by step:

```
18 ÷ 2 = 9 remainder 0
9 ÷ 2 = 4 remainder 1
4 ÷ 2 = 2 remainder 0
2 ÷ 2 = 1 remainder 0
1 ÷ 2 = 0 remainder 1
Binary = 10010
```

Output:

```
Binary = 10010
```

```
public static string DecimalToBinary(int number)
{
    if (number == 0)
        return "0";

    string binary = "";
    while (number > 0)
    {
        int remainder = number % 2;
        binary = remainder + binary; // prepend remainder
        number /= 2;
    }
    return binary;
}
```

## Check if two strings are anagram:

### 🔎 What is an Anagram?

- Two strings are **anagrams** if they contain **exactly the same characters** in any order.
- **Case-insensitive** usually, and sometimes spaces are ignored.
- Examples:
  - `"listen"` and `"silent"` → ✅ Anagrams
  - `"evil"` and `"vile"` → ✅ Anagrams
  - `"hello"` and `"world"` → ❌ Not anagrams

### ✅ Logic to Check Anagram

1. Remove spaces (optional) and convert strings to **same case** (lowercase).
2. Check if **lengths are equal**. If not → Not anagrams.
3. Count the frequency of each character in both strings (using dictionary or array).
4. Compare the frequency counts. If all match → **Anagrams**, else → **Not Anagrams**.

```
public static bool IsAnagram(string s1, string s2)
{
    // Remove spaces and convert to lowercase
    s1 = s1.Replace(" ", "").ToLower();
    s2 = s2.Replace(" ", "").ToLower();

    if (s1.Length != s2.Length)
        return false;

    char[] arr1 = s1.ToCharArray();
    char[] arr2 = s2.ToCharArray();

    Array.Sort(arr1);
    Array.Sort(arr2);

    for (int i = 0; i < arr1.Length; i++)
    {
        if (arr1[i] != arr2[i])
            return false;
    }

    return true;
}
```

**Find the missing number in an integer numbers sequence:**

## ✅ Method 1: Using Sum Formula

- Sum of first n natural numbers: $\text{Sum} = \frac{n(n+1)}{2}$
- Missing number: `Missing = SumOfN - SumOfArray`

### 🔎 Example Run

Input: `{ 1, 2, 4, 5, 6 }`

Output:

```
java

Missing number = 3
```

```java
public static int FindMissingNumber(int[] arr, int n)
{
    int totalSum = n * (n + 1) / 2;
    int arrSum = 0;

    for (int i = 0; i < arr.Length; i++)
        arrSum += arr[i];

    return totalSum - arrSum;
}
```

## ✅ Sample Example

### Example 1: `153`

1. Number of digits → `3` (1, 5, 3)
2. Sum of cubes of digits:

$$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$$

3. Sum equals the original number → ✅ `153 is an Armstrong number`

## Check if the number is an Armstrong number:

An **Armstrong number** (also called a **narcissistic number**) is a number that is equal to the sum of its own digits raised to the power of the number of digits.

**Formula:**

For a number with `n` digits:

```
r

abcd... = a^n + b^n + c^n + d^n + ...
```

Copy code

### 🔢 Logic:

1. Find the **number of digits** `n` in the number.
2. For each digit in the number:
   - Raise it to the power `n`
   - Add to a sum
3. If **sum == original number** → Armstrong number
4. Else → not an Armstrong number

### ✅ Examples

#### Example 1

Number: `153`

- Number of digits = 3
- Sum = $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$ ✅

Output:

```csharp
153 is an Armstrong number
```

```
    public static bool IsArmstrong(int num)
    {
        int originalNum = num;
        int sum = 0;

        // Count digits
        int digits = 0;
        int temp = num;
        while (temp > 0)
        {
            temp /= 10;
            digits++;
        }

        // Calculate sum of digits raised to power of digits
        temp = num;
        while (temp > 0)
        {
            int digit = temp % 10;
            int power = 1;
            for (int i = 0; i < digits; i++)
                power *= digit;

            sum += power;
            temp /= 10;
        }

     return sum == originalNum;
}
```

**Rotate an integer array by k positions (Right Rotation):**

## 🔢 Logic:

1. **Normalize** `k`:
   - If `k` is greater than array length `n`, use `k = k % n`
2. **Split the array** into two parts:
   - Last `k` elements → move to front
   - First `n-k` elements → shift to the right
3. **Combine both parts** to get rotated array

### 🔎 How It Works

1. **Reverse the whole array** → `[7, 6, 5, 4, 3, 2, 1]`
2. **Reverse first k elements** → `[5, 6, 7, 4, 3, 2, 1]`
3. **Reverse remaining elements** → `[5, 6, 7, 1, 2, 3, 4]` ✅ Final rotated array

### 🔎 Example Run

Input:

ini Copy code

```ini
arr = {1,2,3,4,5,6,7}, k = 3
```

Output:

php Copy code

```php
Original array: 1, 2, 3, 4, 5, 6, 7
Rotated array: 5, 6, 7, 1, 2, 3, 4
```

csharp

Copy code

```csharp
using System;

public class Program
{
    public static void Main()
    {
        int[] arr = { 1, 2, 3, 4, 5, 6, 7 };
        int k = 3; // rotate by 3 positions

        Console.WriteLine("Original array: " + string.Join(", ", arr));
        RotateArray(arr, k);
        Console.WriteLine("Rotated array: " + string.Join(", ", arr));
    }

    public static void RotateArray(int[] arr, int k)
    {
        int n = arr.Length;
        k = k % n; // in case k > n

        Reverse(arr, 0, n - 1);       // reverse the whole array
        Reverse(arr, 0, k - 1);       // reverse first k elements
        Reverse(arr, k, n - 1);       // reverse remaining elements
    }

    // Helper method to reverse array elements between indices start and end
    public static void Reverse(int[] arr, int start, int end)
    {
        while (start < end)
        {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }
}
```

## Move zeros to the end of an integer array:

### ✅ Problem Statement

- Input: `[1, 0, 2, 0, 3, 0, 4]`
- Output: `[1, 2, 3, 4, 0, 0, 0]`

**Constraints:**

- Order of non-zero elements **must be preserved**.
- Can be done **in-place** without using extra arrays (optional).

```
public static void MoveZerosToEnd(int[] arr)
{
    int n = arr.Length;
    int count = 0; // index to place non-zero element

    // Move non-zero elements to the front
    for (int i = 0; i < n; i++)
    {
        if (arr[i] != 0)
        {
            arr[count] = arr[i];
            count++;
        }
    }

    // Fill remaining positions with 0
    while (count < n)
    {
        arr[count] = 0;
        count++;
    }
}
```

**Switch statement to perform basic calculator operation:**

```
// Separate function to perform calculation
public static double? Calculate(double num1, double num2, char op)
{
    switch (op)
    {
        case '+':
            return num1 + num2;

        case '-':
            return num1 - num2;

        case '*':
            return num1 * num2;

        case '/':
            if (num2 != 0)
                return num1 / num2;
            else
            {
                Console.WriteLine("Error: Division by zero");
                return null;
            }

        default:
            Console.WriteLine("Invalid operator");
            return null;
    }
}
```

**Reverse a number:**

## Logic to Reverse a Number

Suppose you have an integer `num`. To reverse it:

1. Initialize `reversedNumber = 0`.
2. Loop while `num > 0`:
   - Get the last digit using `digit = num % 10`.
   - Add it to the reversed number: `reversedNumber = reversedNumber * 10 + digit`.
   - Remove the last digit from `num`: `num = num / 10` (integer division).
3. After the loop, `reversedNumber` will be the reversed number.

**Step-by-step Example**:

- Input: `num = 1234`
- Loop 1:
  - digit = 1234 % 10 = 4
  - reversedNumber = 0 * 10 + 4 = 4
  - num = 1234 / 10 = 123
- Loop 2:
  - digit = 123 % 10 = 3
  - reversedNumber = 4 * 10 + 3 = 43
  - num = 123 / 10 = 12
- Loop 3:
  - digit = 12 % 10 = 2
  - reversedNumber = 43 * 10 + 2 = 432
  - num = 12 / 10 = 1
- Loop 4:
  - digit = 1 % 10 = 1
  - reversedNumber = 432 * 10 + 1 = 4321
  - num = 1 / 10 = 0

**Output**: `4321`

```
public static int ReverseNumber(int n)
{
    int reversed = 0;

    while (n != 0)
    {
        int digit = n % 10;         // get last digit
        reversed = reversed * 10 + digit; // append digit
        n /= 10;                    // remove last digit
    }

    return reversed;
}
```

## Find the largest of 3 numbers:

### Logic to Find the Largest Number

Suppose you have three numbers: `a`, `b`, and `c`.

1. Initialize a variable `largest`.
2. Compare `a` with `b`:
   - If `a > b`, then `largest = a`; else `largest = b`.
3. Compare `largest` with `c`:
   - If `c > largest`, then `largest = c`.
4. After comparisons, `largest` will hold the largest number.

### Step-by-step Example

- Input: `a = 10, b = 25, c = 15`
- Compare a and b:
  - 10 > 25? No → `largest = 25`
- Compare largest and c:
  - 25 > 15? Yes → `largest = 25`
- **Output**: `25`

Another example:

- Input: `a = 7, b = 12, c = 20`
- Compare a and b: 7 > 12? No → `largest = 12`
- Compare largest and c: 12 > 20? No → `largest = 20`
- **Output**: `20`

```
// Separate function to find largest of 3 numbers
public static int FindLargest(int a, int b, int c)
{
    if (a >= b && a >= c)
        return a;
    else if (b >= a && b >= c)
        return b;
    else
        return c;
}
```

**C# function to find the maximum consecutive ones in a binary array:**

## Problem Statement

Given a binary array, e.g., `[1,1,0,1,1,1]`, find the **maximum number of consecutive 1s**.

- In `[1,1,0,1,1,1]`, the consecutive 1s are: `[1,1]` and `[1,1,1]`
- Maximum consecutive 1s = `3`

### Logic

1. Initialize two variables:
   - `maxCount = 0` → stores the maximum consecutive 1s found so far
   - `currentCount = 0` → stores the current streak of consecutive 1s
2. Loop through the array:
   - If the current element is `1`, increment `currentCount` and update `maxCount` if needed (`maxCount = Math.Max(maxCount, currentCount)`)
   - If the current element is `0`, reset `currentCount` to `0`
3. After the loop, `maxCount` will hold the maximum consecutive 1s

### Step-by-step Example

Array: `[1,1,0,1,1,1]`

- i=0 → 1 → currentCount = 1, maxCount = 1
- i=1 → 1 → currentCount = 2, maxCount = 2
- i=2 → 0 → currentCount = 0, maxCount = 2
- i=3 → 1 → currentCount = 1, maxCount = 2
- i=4 → 1 → currentCount = 2, maxCount = 2
- i=5 → 1 → currentCount = 3, maxCount = 3

**Output:** `3`

```
public static int FindMaxConsecutiveOnes(int[] nums)
{
    int maxCount = 0, currentCount = 0;

    foreach (int num in nums)
    {
        if (num == 1)
        {
            currentCount++;

            if (currentCount > maxCount)
            {
                maxCount = currentCount;
            }
        }
        else
        {
            currentCount = 0; // reset on 0
        }
    }

    return maxCount;
}
```

**A bubble sort implementation to sort an integer array in ascending order:**

## ✅ What it does

- The function `SortArray(int[] arr)` sorts an integer array in **ascending order**.
- It repeatedly compares **adjacent elements** and swaps them if they are in the wrong order.
- This process continues until the largest elements "bubble up" to the end of the array.

## ✅ Step-by-step:

1. Outer loop ( `for (int i = 0; i < n - 1; i++)` ) → runs `n-1` passes.
2. Inner loop ( `for (int j = 0; j < n - i - 1; j++)` ) → compares adjacent pairs.
3. If `arr[j] > arr[j+1]`, swap them.
4. After each pass, the largest element moves to its correct position.

## ✅ Example

Input:

```csharp
[5, 3, 8, 4, 2]
```

Pass 1 → `[3, 5, 4, 2, 8]`
Pass 2 → `[3, 4, 2, 5, 8]`
Pass 3 → `[3, 2, 4, 5, 8]`
Pass 4 → `[2, 3, 4, 5, 8]`

Output:

```csharp
[2, 3, 4, 5, 8]
```

```
public static void SortArray(int[] arr)
{
    int n = arr.Length;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                // Swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

**A bubble sort implementation to sort an integer array in descending order:**

```
public static void SortDescending(int[] arr)
{
    int n = arr.Length;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] < arr[j + 1]) // swap if left < right
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

**Merge 2 sorted arrays:**

## Problem Statement

You have two **sorted arrays**, for example:

```
text

arr1 = [1, 3, 5]
arr2 = [2, 4, 6]
```

We want to merge them into a **single sorted array**: `[1, 2, 3, 4, 5, 6]`.

## Logic

1. Initialize three pointers:
   - `i = 0` → pointer for `arr1`
   - `j = 0` → pointer for `arr2`
   - `k = 0` → pointer for `mergedArray`
2. Compare `arr1[i]` and `arr2[j]`:
   - If `arr1[i] <= arr2[j]`, put `arr1[i]` in `mergedArray[k]` and increment `i` and `k`
   - Else, put `arr2[j]` in `mergedArray[k]` and increment `j` and `k`
3. After one array is exhausted, copy the remaining elements from the other array into `mergedArray`.
4. Return `mergedArray`.

**Time Complexity:** `O(n + m)`
**Space Complexity:** `O(n + m)`

### Step-by-step Example

`arr1 = [1, 3, 5]`
`arr2 = [2, 4, 6]`

- Compare 1 and 2 → 1 is smaller → merged = [1]
- Compare 3 and 2 → 2 is smaller → merged = [1, 2]
- Compare 3 and 4 → 3 is smaller → merged = [1, 2, 3]
- Compare 5 and 4 → 4 is smaller → merged = [1, 2, 3, 4]
- Compare 5 and 6 → 5 is smaller → merged = [1, 2, 3, 4, 5]
- Remaining 6 → merged = [1, 2, 3, 4, 5, 6]

```
public static int[] MergeSortedArrays(int[] arr1, int[] arr2)
{
    int i = 0, j = 0, k = 0;
    int[] result = new int[arr1.Length + arr2.Length];

    // Compare elements from both arrays
    while (i < arr1.Length && j < arr2.Length)
    {
        if (arr1[i] <= arr2[j])
        {
            result[k++] = arr1[i++];
        }
        else
        {
            result[k++] = arr2[j++];
        }
    }

    // Copy remaining elements of arr1
    while (i < arr1.Length)
    {
        result[k++] = arr1[i++];
    }

    // Copy remaining elements of arr2
    while (j < arr2.Length)
    {
        result[k++] = arr2[j++];
    }

    return result;
}
```

**Find the maximum sum of a subarray of an integer array:**

## Problem Statement

Given an integer array, find the **maximum sum of a contiguous subarray**.

Example:

```
text

arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
```

Maximum sum subarray = `[4, -1, 2, 1]` → sum = `6`

## Logic (Kadane's Algorithm)

1. Initialize two variables:
   - `maxSoFar = arr[0]` → stores the maximum sum found so far
   - `currentMax = arr[0]` → stores the maximum sum ending at the current position
2. Traverse the array from index `1` to `n-1`:
   - `currentMax = Math.Max(arr[i], currentMax + arr[i])` → either start a new subarray at `arr[i]` or extend the current subarray
   - `maxSoFar = Math.Max(maxSoFar, currentMax)` → update the maximum sum found so far
3. After the loop, `maxSoFar` will contain the maximum subarray sum.

**Time Complexity:** `O(n)`

**Space Complexity:** `O(1)`

### Step-by-step Example

Array: `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`

| Index | arr[i] | currentMax | maxSoFar |
|---|---|---|---|
| 0 | -2 | -2 | -2 |
| 1 | 1 | max(1, -2+1)=1 | max(-2,1)=1 |
| 2 | -3 | max(-3,1-3)=-2 | max(1,-2)=1 |
| 3 | 4 | max(4,-2+4)=4 | max(1,4)=4 |
| 4 | -1 | max(-1,4-1)=3 | max(4,3)=4 |
| 5 | 2 | max(2,3+2)=5 | max(4,5)=5 |
| 6 | 1 | max(1,5+1)=6 | max(5,6)=6 |
| 7 | -5 | max(-5,6-5)=1 | max(6,1)=6 |
| 8 | 4 | max(4,1+4)=5 | max(6,5)=6 |

**Maximum sum = 6**

```
public static int MaxSubArraySum(int[] arr)
{
    int maxSoFar = arr[0];     // Best sum found so far
    int currentMax = arr[0];   // Max sum ending at current index

    for (int i = 1; i < arr.Length; i++)
    {
        // Either extend the current subarray OR start a new subarray
        currentMax = Math.Max(arr[i], currentMax + arr[i]);

        // Update global max if needed
        maxSoFar = Math.Max(maxSoFar, currentMax);
    }

    return maxSoFar;
}
```

## Sort an integer array by frequency of each number:

### Problem Statement

Given an integer array, sort it **based on the frequency of elements** in **ascending order**.

- If two numbers have the same frequency, sort by the number value itself (optional, usually ascending).

Example:

```
text                                                    Copy code

arr = [4, 5, 6, 5, 4, 3]
```

- Frequencies: 4 → 2, 5 → 2, 6 → 1, 3 → 1
- Sorting by frequency ascending: 6, 3, 4, 4, 5, 5

**Output:** `[6, 3, 4, 4, 5, 5]`

### Logic

1. Count the **frequency of each element** using a dictionary ( `Dictionary<int, int>` in C#).
2. Sort the array based on two criteria:
   - **Primary:** frequency (ascending)
   - **Secondary:** value of the element (ascending) for tie-break
3. Reconstruct the array in sorted order.

### Step-by-step Example

Array: `[4, 5, 6, 5, 4, 3]`

- Count frequencies:

```
4 → 2
5 → 2
6 → 1
3 → 1
```

- Sort by frequency (ascending) → `[6, 3, 4, 4, 5, 5]`

csharp

```csharp
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    static void Main()
    {
        int[] arr = { 4, 5, 6, 5, 4, 3, 4 };

        int[] result = SortByFrequency(arr);

        Console.WriteLine("Array sorted by frequency:");
        Console.WriteLine(string.Join(" ", result));
    }
```

```csharp
  public static int[] SortByFrequency(int[] arr)
  {
      // Count frequencies
      Dictionary<int, int> freq = new Dictionary<int, int>();
      foreach (int num in arr)
      {
          if (freq.ContainsKey(num))
              freq[num]++;
          else
              freq[num] = 1;
      }

      // Sort by frequency (desc), then by value (asc)
      var sorted = arr
          .OrderByDescending(x => freq[x])  // High frequency first
          .ThenBy(x => x)                   // If same frequency, sort by value
          .ToArray();

      return sorted;
  }
```

## Remove vowels from a string:

### Problem Statement

Given a string, remove all **vowels** ( `a, e, i, o, u` – both uppercase and lowercase) and return the modified string.

Example:

```text
Input: "Hello World"
Output: "Hll Wrld"
```

Copy code

## Logic

1. Define a set of vowels: `a, e, i, o, u` (and uppercase: `A, E, I, O, U`).
2. Loop through each character of the string:
   - If the character is **not a vowel**, add it to a new string or StringBuilder.
   - If it is a vowel, skip it.
3. Return the new string without vowels.

### Step-by-step Example

Input: `"Programming"`

- Characters: P → keep
- r → keep
- o → vowel → skip
- g → keep
- r → keep
- a → vowel → skip
- m → keep
- m → keep
- i → vowel → skip
- n → keep
- g → keep

**Output:** `"Prgrmmng"`

✅ **Time Complexity:** `O(n)` (n = length of string)

✅ **Space Complexity:** `O(n)` (for new string)

```csharp
// ✅ Remove all vowels (a, e, i, o, u - both upper & lower case)
public static string RemoveVowels(string str)
{
    string vowels = "aeiouAEIOU";
    StringBuilder sb = new StringBuilder();

    foreach (char c in str)
    {
        if (!vowels.Contains(c))
            sb.Append(c);
    }

    return sb.ToString();
}
```

**Remove specified character from a string:**

## Problem Statement

Given a string and a character `ch`, remove all occurrences of `ch` from the string.

Example:

```text
Input: "hello world", ch = 'l'
Output: "heo word"
```

### Logic

1. Initialize a new string or a `StringBuilder`.
2. Loop through each character of the string:
   - If the character is **not equal** to the specified character, add it to the new string.
   - Otherwise, skip it.
3. Return the new string.

### Step-by-step Example

Input: `"programming"`, ch = `'g'`

- Characters: p → keep
- r → keep
- o → keep
- g → remove
- r → keep
- a → keep
- m → keep
- m → keep
- i → keep
- n → keep
- g → remove

**Output:** `"prorammin"`

```
// ✅ Remove a specific character
public static string RemoveCharacter(string str, char ch)
{
    StringBuilder sb = new StringBuilder();

    foreach (char c in str)
    {
        if (c != ch)
            sb.Append(c);
    }

    return sb.ToString();
}
```

**Capitalize the first letter of a word in a sentence:**

## Problem Statement

Given a sentence, capitalize the **first letter of each word**.

Example:

```
text

Input: "hello world from csharp"
Output: "Hello World From Csharp"
```

## Logic

1. Split the sentence into words using space as a separator.
2. For each word:
   - Take the first character and convert it to uppercase.
   - Concatenate it with the rest of the word (unchanged).
3. Join all words back together with a space.
4. Return the modified sentence.

### Step-by-step Example

Input: `"good morning"`

- Split → `["good", "morning"]`
- Capitalize each word:
  - `"good"` → `"Good"`
  - `"morning"` → `"Morning"`
- Join → `"Good Morning"`

```
public static string CapitalizeWords(string sentence)
{
    if (string.IsNullOrWhiteSpace(sentence))
        return sentence;

    StringBuilder sb = new StringBuilder(sentence.Length);
    bool capitalizeNext = true;

    foreach (char c in sentence)
    {
        if (char.IsWhiteSpace(c))
        {
            sb.Append(c);
            capitalizeNext = true; // next character should be capitalized
        }
        else if (capitalizeNext)
        {
            sb.Append(char.ToUpper(c));
            capitalizeNext = false;
        }
        else
        {
            sb.Append(char.ToLower(c));
        }
    }

    return sb.ToString();
}
```

## Longest substring in a string without repeating characters:

### Problem Statement

Given a string `s`, find the length of the longest substring without repeating characters.

**Example:**

- Input: `"abcabcbb"`
- Output: `3` → longest substring is `"abc"`
- Input: `"bbbbb"`
- Output: `1` → longest substring is `"b"`
- Input: `"pwwkew"`
- Output: `3` → longest substring is `"wke"`

### Logic

1. Use **sliding window** technique with two pointers (`start` and `end`) to maintain a substring without repeating characters.
2. Use a **dictionary or hash map** to store the last index of each character.
3. Move the `end` pointer forward one character at a time:
   - If the character is **already in the dictionary and its index is >= start**, update `start` to `index + 1` (to skip the repeating character).
   - Update the dictionary with the current character index.
   - Calculate the length of the current substring (`end - start + 1`) and keep track of the **maximum length**.
4. Continue until the end of the string.

**Time Complexity:** `O(n)` (each character is visited once)
**Space Complexity:** `O(min(n, charset))` (for storing characters in dictionary)

### Step-by-Step Example

**Input:** `"abcabcbb"`

| Step | Char | start | end | Current Substring | maxLength | Map |
|---|---|---|---|---|---|---|
| 0 | a | 0 | 0 | "a" | 1 | {a:0} |
| 1 | b | 0 | 1 | "ab" | 2 | {a:0, b:1} |
| 2 | c | 0 | 2 | "abc" | 3 | {a:0, b:1, c:2} |
| 3 | a | 1 | 3 | "bca" | 3 | {a:3, b:1, c:2} |
| 4 | b | 2 | 4 | "cab" | 3 | {a:3, b:4, c:2} |
| 5 | c | 3 | 5 | "abc" | 3 | {a:3, b:4, c:5} |
| 6 | b | 5 | 6 | "cb" | 3 | {a:3, b:6, c:5} |
| 7 | b | 7 | 7 | "b" | 3 | {a:3, b:7, c:5} |

✅ **Result:** `3` → `"abc"`

```csharp
public static int LongestSubstringWithoutRepeating(string s)
{
    HashSet<char> set = new HashSet<char>();
    int left = 0, maxLength = 0;

    for (int right = 0; right < s.Length; right++)
    {
        // If char already in set, move left until it's removed
        while (set.Contains(s[right]))
        {
            set.Remove(s[left]);
            left++;
        }

        // Add current char
        set.Add(s[right]);

        // Update max length
        maxLength = Math.Max(maxLength, right - left + 1);
    }

    return maxLength;
}
```

## Count words in a sentence:

### Problem Statement

Given a sentence (string), count the number of words in it.

- Words are usually separated by **spaces**.
- Multiple consecutive spaces should not count as extra words.

**Example:**

- Input: `"Hello world"` → Output: `2`
- Input: `" This is C# code "` → Output: `4`
- Input: `""` → Output: `0`

### Logic

1. **Trim** the string to remove leading and trailing spaces.
2. **Check if string is empty** after trimming → if yes, return 0.
3. **Split** the string by spaces using `Split` method.
   - Use `StringSplitOptions.RemoveEmptyEntries` to ignore multiple spaces.
4. Return the **length** of the resulting array → this is the word count.

```
public static int CountWords(string sentence)
{
    if (string.IsNullOrWhiteSpace(sentence))
        return 0;

    // Split by spaces, remove empty entries
    string[] words = sentence.Split(new char[] { ' ', '\t', '\n' },
                                    StringSplitOptions.RemoveEmptyEntries);

    return words.Length;
}
```

## Remove duplicate numbers from an integer array using HashSet:

### Problem Statement

Given an integer array, remove duplicates so that each number appears only once and return the result.

**Example:**

- Input: `[1, 2, 2, 3, 4, 4, 5]` → Output: `[1, 2, 3, 4, 5]`
- Input: `[5, 5, 5, 5]` → Output: `[5]`
- Input: `[1, 2, 3]` → Output: `[1, 2, 3]`

### Logic

1. Use a **HashSet** to store unique numbers (HashSet automatically removes duplicates).
2. Loop through the array and add each element to the HashSet.
3. Convert the HashSet back to an array if needed.

**Time Complexity:** `O(n)` (each number added once)
**Space Complexity:** `O(n)` (for HashSet)

```
public static int[] RemoveDuplicates(int[] arr)
{
    HashSet<int> set = new HashSet<int>();
    foreach (int num in arr)
    {
        set.Add(num);
    }
    return new List<int>(set).ToArray();
}
```

## Remove duplicate numbers from an integer array using Dictionary:

### Problem Statement

Given an integer array, remove duplicate numbers using a dictionary so that each number appears only once.

**Example:**

- Input: `[1, 2, 2, 3, 4, 4, 5]` → Output: `[1, 2, 3, 4, 5]`
- Input: `[5, 5, 5, 5]` → Output: `[5]`
- Input: `[1, 2, 3]` → Output: `[1, 2, 3]`

### Logic

1. Create a **Dictionary<int, bool>** to track numbers already added.
2. Loop through the array:
   - If the number is **not in the dictionary**, add it to the dictionary and to a **result list**.
   - If the number is already in the dictionary, skip it.
3. Convert the result list to an array if needed.

**Time Complexity:** `O(n)`
**Space Complexity:** `O(n)`

```
public static int[] RemoveDuplicatesUsingDictionary(int[] arr)
{
    Dictionary<int, bool> dict = new Dictionary<int, bool>();
    List<int> result = new List<int>();

    foreach (int num in arr)
    {
        if (!dict.ContainsKey(num))
        {
            dict[num] = true;   // mark as seen
            result.Add(num);    // add to result
        }
    }

    return result.ToArray();
}
```

## Reverse words in a sentence:

### Problem Statement

Given a sentence (string), reverse the **order of words** while keeping the words themselves intact.

**Example:**

- Input: `"Hello world"` → Output: `"world Hello"`
- Input: `"C# is awesome"` → Output: `"awesome is C#"`
- Input: `" This is fun "` → Output: `"fun is This"`

### Logic

1. **Trim** the string to remove leading and trailing spaces.
2. **Split** the string into words using space as a separator and **ignore multiple spaces**.
3. **Reverse** the array of words.
4. **Join** the words back into a single string using a space.

**Time Complexity:** `O(n)`
**Space Complexity:** `O(n)`

```
public class Program
{
    public static void Main()
    {
        Console.Write("Enter a sentence: ");
        string sentence = Console.ReadLine();

        string reversed = ReverseWords(sentence);
        Console.WriteLine("Reversed words: " + reversed);
    }

    public static string ReverseWords(string str)
    {
        string[] words = str.Split(' '); // split words by space
        Array.Reverse(words);            // reverse the array of words
        return string.Join(" ", words);  // join words back into a string
    }
}
```

```
public static string ReverseWords(string sentence)
{
    if (string.IsNullOrWhiteSpace(sentence))
        return "";

    // Split words and remove empty entries caused by multiple spaces
    string[] words = sentence.Trim().Split(' ', StringSplitOptions.RemoveEmptyEntries);

    // Reverse the words array
    Array.Reverse(words);

    // Join the words back into a sentence
    return string.Join(" ", words);
}
```

## Check if the brackets are balanced or not:

### Problem Statement

Given a string containing brackets: `()`, `{}`, `[]`, determine if the brackets are **balanced**.

- Balanced means every opening bracket has a corresponding closing bracket in the correct order.

**Example:**

- Input: `"()[]{}"` → Output: `true`
- Input: `"([{}])"` → Output: `true`
- Input: `"([)]"` → Output: `false`
- Input: `"(((("` → Output: `false`

### Logic

1. Use a **stack** to keep track of opening brackets.
2. Loop through each character in the string:
    - If the character is an **opening bracket** (`(`, `{`, `[`), push it onto the stack.
    - If it is a **closing bracket** (`)`, `}`, `]`):
        - Check if the stack is empty → if yes, brackets are **not balanced**.
        - Pop the top element from the stack and check if it **matches** the closing bracket.
            - `(` matches `)`
            - `{` matches `}`
            - `[` matches `]`
        - If it doesn't match → brackets are **not balanced**.
3. After processing all characters, if the stack is **empty**, brackets are balanced. Otherwise, they are not.

**Time Complexity:** `O(n)`
**Space Complexity:** `O(n)`

### Step-by-Step Example

**Input:** `"([{}])"`

| Char | Stack | Action |
|---|---|---|
| ( | ( | push '(' |
| [ | ([ | push '[' |
| { | ([{ | push '{' |
| } | ([ | pop '{', match '}' |
| ] | ( | pop '[', match ']' |
| ) | empty | pop '(', match ')' |

```
public static bool AreBracketsBalanced(string str)
{
    Stack<char> stack = new Stack<char>();

    foreach (char ch in str)
    {
        if (ch == '(' || ch == '{' || ch == '[')
        {
            stack.Push(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']')
        {
            if (stack.Count == 0)
                return false;

            char last = stack.Pop();

            if (!IsMatchingPair(last, ch))
                return false;
        }
    }

    return stack.Count == 0;
}
```

```
private static bool IsMatchingPair(char open, char close)
{
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}
```

```
static void Main()
{
    string input = "{[()]}"; // Example input
    bool isBalanced = AreBracketsBalanced(input);
    Console.WriteLine(isBalanced ? "Balanced" : "Not Balanced");
}
```

**Demonstrate the use cases of ref, out, and in parameters in C# with examples:**

## 1. `ref` Parameter

### Logic

- Passes a variable **by reference**, allowing the method to **read and modify** the caller's variable.
- The variable **must be initialized** before passing it to the method.

### Use Case

- When you want a method to **update the value** of a variable and reflect that change outside the method.

## 2. `out` Parameter

### Logic

- Passes a variable **by reference** but **does not require initialization** before passing.
- The method **must assign a value** to the `out` parameter before returning.

### Use Case

- When a method needs to **return multiple values**, especially **one or more optional results**.

## 3. `in` Parameter

### Logic

- Passes a variable **by reference** but **read-only** inside the method.
- Prevents modification but avoids the overhead of copying large structs.

### Use Case

- When you want to **pass large structs efficiently** without copying, and **ensure the method does not modify them**.

```
public class Program
{
    public static void Main()
    {
        int a = 5;
        int b = 10;
        int c;

        Console.WriteLine($"Before function: a = {a}, b = {b}");

        ProcessNumbers(in a, ref b, out c);

        Console.WriteLine($"After function: a = {a}, b = {b}, c = {c}");
    }

    // Single function demonstrating in, ref, and out
    public static void ProcessNumbers(in int x, ref int y, out int z)
    {
        // x is 'in' parameter → read-only
        Console.WriteLine($"Inside function - in parameter x = {x}");

        // y is 'ref' parameter → can read and modify
        y += 5;

        // z is 'out' parameter → must be assigned inside function
        z = x + y;
    }
}
```

### 🔎 How It Works

1. `in` **parameter (`x`):**
   - Read-only inside the function.
   - Cannot be modified.
2. `ref` **parameter (`y`):**
   - Read/write inside the function.
   - Changes are reflected in the caller.
3. `out` **parameter (`z`):**
   - Must be assigned inside the function.
   - Used to return additional results.

**Demonstrate C# code for exception handling:**

```
public class Program
{
    public static void Main()
    {
        try
        {
            Console.Write("Enter a number: ");
            int num = Convert.ToInt32(Console.ReadLine());

            int result = 100 / num; // may cause DivideByZeroException
            Console.WriteLine($"Result: {result}");
        }
        catch (DivideByZeroException)
        {
            Console.WriteLine("Error: Cannot divide by zero");
        }
        catch (FormatException)
        {
            Console.WriteLine("Error: Invalid input, please enter a number");
        }
        finally
        {
            Console.WriteLine("Program completed.");
        }
    }
}
```

**Create a class with properties and method. Use it inside a Main method.**

```
using System;

public class Person
{
    // Properties
    public string Name { get; set; }
    public int Age { get; set; }

    // Method
    public void Introduce()
    {
        Console.WriteLine($"Hello! My name is {Name} and I am {Age} years old.");
    }
}
```

```
public class Program
{
    public static void Main()
    {
        // Create an object of Person
        Person person1 = new Person();
        person1.Name = "Alice";
        person1.Age = 25;

        // Call method
        person1.Introduce();
    }
}
```

**Print a star pattern pyramid:**

```
public static void Main()
{
    Console.Write("Enter number of rows: ");
    int rows = Convert.ToInt32(Console.ReadLine());

    for (int i = 1; i <= rows; i++)
    {
        // Print spaces
        for (int j = 1; j <= rows - i; j++)
        {
            Console.Write(" ");
        }

        // Print stars
        for (int k = 1; k <= 2 * i - 1; k++)
        {
            Console.Write("*");
        }

        Console.WriteLine(); // move to next row
    }
}
```

### 🔎 Example Run

**Input:**

```
sql

Enter number of rows: 5
```

**Output:**

```
markdown

    *
   ***
  *****
 *******
*********
```

## Count frequency of each word in a sentence:

### Problem Statement

Given a sentence, count how many times each word appears.

- Words are separated by spaces.
- Ignore case and punctuation for accurate counting if needed.

**Example:**

- Input: `"This is is a test test"`
- Output:

```rust

This -> 1
is -> 2
a -> 1
test -> 2
```

### Logic

1. **Trim** the sentence to remove leading and trailing spaces.
2. **Split** the sentence into words using space as a separator and **ignore empty entries**.
3. Use a **Dictionary<string, int>** to store each word and its count:
   - If the word exists in the dictionary, **increment its count**.
   - Otherwise, **add it with count 1**.
4. Optionally, **ignore case** by converting all words to lowercase.

**Time Complexity:** `O(n)` where `n` is the number of words
**Space Complexity:** `O(m)` where `m` is the number of unique words

```
public static Dictionary<string, int> CountWordFrequency(string sentence)
{
    Dictionary<string, int> wordCount = new Dictionary<string, int>();

    if (string.IsNullOrWhiteSpace(sentence))
        return wordCount;

    // Normalize sentence (remove punctuation, make lowercase)
    char[] delimiters = { ' ', '\t', '\n', ',', '.', '!', '?', ';', ':' };
    string[] words = sentence.ToLower().Split(delimiters, StringSplitOptions.RemoveEmptyEntries);

    foreach (string word in words)
    {
        if (wordCount.ContainsKey(word))
            wordCount[word]++;
        else
            wordCount[word] = 1;
    }

    return wordCount;
}
```

## Demonstrate C# code to serialize and deserialize an object:

### Problem Statement

- **Serialization**: Convert an object into a format (e.g., JSON or XML) that can be stored or transmitted.
- **Deserialization**: Convert the stored or transmitted format back into an object.

**Use Case:** Saving settings to a file, sending objects over a network, storing objects in a database.

### Logic

1. Create a class (object) to serialize.
2. Use `JsonSerializer` (from `System.Text.Json`) for JSON serialization.
3. Serialize the object to a **string or file**.
4. Deserialize the string or file back to the **original object type**.

**Notes:**

- Ensure the class has **public properties**.
- JSON is widely used, but XML serialization is also possible using `XmlSerializer`.

csharp

```csharp
using System;
using System.Text.Json;

public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

Copy code

```csharp
public class Program
{
    public static void Main()
    {
        // Create an object
        Person person = new Person { Name = "Alice", Age = 25 };

        // Serialize object to JSON
        string jsonString = JsonSerializer.Serialize(person);
        Console.WriteLine("Serialized JSON:");
        Console.WriteLine(jsonString);

        // Deserialize JSON back to object
        Person deserializedPerson = JsonSerializer.Deserialize<Person>(jsonString);
        Console.WriteLine("\nDeserialized Object:");
        Console.WriteLine($"Name: {deserializedPerson.Name}, Age: {deserializedPerson.Age}");
    }
}
```

## 🔎 How It Works

1. **Serialize** → Convert C# object to JSON string using `JsonSerializer.Serialize()`.
2. **Deserialize** → Convert JSON string back to C# object using `JsonSerializer.Deserialize<T>()`.

## 🔎 Example Output

css

```
Serialized JSON:
{"Name":"Alice","Age":25}

Deserialized Object:
Name: Alice, Age: 25
```

**Create a Singleton class and validate it:**

## Problem Statement

- **Singleton Pattern** ensures a class has **only one instance** and provides a **global point of access** to it.
- Use case: Logging, Configuration Manager, Database Connection, etc.

```
public class Singleton
{
    // Private static variable to hold the single instance
    private static Singleton _instance;

    // Private constructor to prevent direct instantiation
    private Singleton()
    {
        Console.WriteLine("Singleton instance created.");
    }

    // Public static method to get the instance
    public static Singleton GetInstance()
    {
        if (_instance == null)
        {
            _instance = new Singleton();
        }
        return _instance;
    }

    // Example method
    public void ShowMessage()
    {
        Console.WriteLine("Hello from Singleton!");
    }
}
```

```
public class Program
{
    public static void Main()
    {
        // Get the singleton instance multiple times
        Singleton s1 = Singleton.GetInstance();
        Singleton s2 = Singleton.GetInstance();

        // Call method
        s1.ShowMessage();

        // Validate that both instances are same
        if (s1 == s2)
        {
            Console.WriteLine("Both instances are the same. Singleton works!");
        }
        else
        {
            Console.WriteLine("Different instances. Singleton failed.");
        }
    }
}
```

### Output

python Copy code

```
Singleton instance created!
Hello from Singleton!
Hello from Singleton!
True
```

- Notice "**Singleton instance created!**" is printed **only once**, confirming single instantiation.

Alternative approach to validate Singleton class inside Main method:

```
class Program
{
    static void Main()
    {
        // Access Singleton instance multiple times
        Singleton s1 = Singleton.Instance;
        s1.ShowMessage();

        Singleton s2 = Singleton.Instance;
        s2.ShowMessage();

        // Validate that both references point to the same instance
        Console.WriteLine(object.ReferenceEquals(s1, s2)); // Output: True
    }
}
```

**Thread-Safe Singleton Using Lazy<T> (Recommended):**

## Logic

1. **Private Constructor**: Prevents external instantiation.
2. **Private static variable**: Holds the single instance of the class.
3. **Public static property or method**: Returns the instance.
4. **Thread-safety**: Use locking or `Lazy<T>` to ensure thread-safe instantiation in multithreaded environments.

## Step-by-Step Explanation

1. **Private constructor** prevents `new Singleton()` from being called outside the class.
2. **Lazy<T>** ensures:
   - Only one instance is created.
   - Thread-safe without explicit locks.
3. **Singleton.Instance** provides a global access point.
4. **Validation**:

csharp

```
object.ReferenceEquals(s1, s2) // True → both refer to the same instance
```

csharp

Copy code

```csharp
using System;

class Singleton
{
    // Lazy<T> ensures thread-safe, lazy initialization
    private static readonly Lazy<Singleton> instance = new Lazy<Singleton>(() => new Singleton());

    // Private constructor prevents external instantiation
    private Singleton()
    {
        Console.WriteLine("Singleton instance created!");
    }

    // Public property to provide global access
    public static Singleton Instance
    {
        get
        {
            return instance.Value;
        }
    }

    // Sample method
    public void ShowMessage()
    {
        Console.WriteLine("Hello from Singleton!");
    }
}
```

## Demonstrate the usage of extension methods:

### Problem Statement

- **Extension methods** allow you to **add new methods** to existing types (classes, structs, interfaces) **without modifying their source code**.
- Use case: Adding utility methods to `string`, `int`, `List<T>`, or custom classes.

### Logic

1. Create a **static class** to hold the extension method.
2. The method itself must be **static**.
3. Use the `this` **keyword** as the **first parameter**, which specifies the type being extended.
4. Call the extension method as if it were a **regular instance method** of the type.

#### Key Points

- Extension methods **cannot access private members** of the class they extend.
- They are **resolved at compile time**, not runtime.
- If an instance method has the same signature as an extension method, the **instance method takes priority**.

```
public static class StringExtensions
{
    // Extension method to count words in a string
    public static int WordCount(this string str)
    {
        if (string.IsNullOrWhiteSpace(str))
            return 0;
        return str.Split(' ').Length;
    }
}
```

```
class Program
{
    static void Main()
    {
        string sentence = "Hello world from C# extension methods!";
        int count = sentence.WordCount();  // Call as instance method
        Console.WriteLine($"Word count: {count}"); // Output: Word count: 6
    }
}
```

```csharp
public static class IntExtensions
{
    // Extension method to check if a number is even
    public static bool IsEven(this int number)
    {
        return number % 2 == 0;
    }

    // Extension method to check if a number is odd
    public static bool IsOdd(this int number)
    {
        return number % 2 != 0;
    }
}
```

```csharp
public class Program
{
    public static void Main()
    {
        int num1 = 10;
        int num2 = 7;

        // Calling the extension methods
        Console.WriteLine($"{num1} is even? {num1.IsEven()}");
        Console.WriteLine($"{num2} is odd? {num2.IsOdd()}");
    }
}
```

**Demonstrate C# implementation of CRUD operations on Student entity:**

## Logic

1. Use a **List<T>** to store objects (simulates a database).
2. Implement **methods** for each operation:
   - **Create**: Add an object to the list.
   - **Read**: Fetch objects from the list.
   - **Update**: Find an object by ID and modify it.
   - **Delete**: Find an object by ID and remove it.
3. Use a **unique identifier** (like `Id`) to locate objects.

## C# Implementation

csharp

```csharp
using System;
using System.Collections.Generic;
using System.Linq;

// Sample class
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

```
class Program
{
    // In-memory database
    static List<Student> students = new List<Student>();

    // --- CREATE ---
    static void AddStudent(Student student)
    {
        students.Add(student);
        Console.WriteLine($"Student {student.Name} added.");
    }

    // --- READ ---
    static void GetAllStudents()
    {
        Console.WriteLine("\nAll Students:");
        foreach (var s in students)
        {
            Console.WriteLine($"Id: {s.Id}, Name: {s.Name}");
        }
    }
```

```
    static Student GetStudentById(int id)
    {
        return students.FirstOrDefault(s => s.Id == id);
    }

    // --- UPDATE ---
    static void UpdateStudent(int id, string newName)
    {
        var student = GetStudentById(id);
        if (student != null)
        {
            student.Name = newName;
            Console.WriteLine($"Student {id} updated to {newName}.");
        }
        else
        {
            Console.WriteLine($"Student {id} not found.");
        }
    }
```

```
    // --- DELETE ---
    static void DeleteStudent(int id)
    {
        var student = GetStudentById(id);
        if (student != null)
        {
            students.Remove(student);
            Console.WriteLine($"Student {id} deleted.");
        }
        else
        {
            Console.WriteLine($"Student {id} not found.");
        }
    }
```

```
    static void Main()
    {
        // Create
        AddStudent(new Student { Id = 1, Name = "John" });
        AddStudent(new Student { Id = 2, Name = "Alice" });

        // Read
        GetAllStudents();

        // Update
        UpdateStudent(1, "John Smith");
        GetAllStudents();

        // Delete
        DeleteStudent(2);
        GetAllStudents();
    }
}
```

### Step-by-Step Example

1. **AddStudent**: Adds `John` and `Alice` → list has 2 items.
2. **GetAllStudents**: Prints both students.
3. **UpdateStudent**: Updates `John` to `John Smith`.
4. **DeleteStudent**: Removes `Alice` → list now has 1 item (`John Smith`).

**Output:**

yaml

Copy code

```
Student John added.
Student Alice added.

All Students:
Id: 1, Name: John
Id: 2, Name: Alice
Student 1 updated to John Smith.

All Students:
Id: 1, Name: John Smith
Id: 2, Name: Alice
Student 2 deleted.

All Students:
Id: 1, Name: John Smith
```

### Key Points

- This example uses **List<T>** to simulate a database.
- In real-world apps, CRUD operations are implemented using **Entity Framework Core**, **ADO.NET**, or **API calls**.
- Using `LINQ` (`FirstOrDefault`) makes **searching and updating** easy.

## Demonstrate asynchronous operation on I/O bound operation:

### Problem Statement

- I/O-bound operations include tasks like **file reading/writing**, **HTTP requests**, or **database calls**.
- Asynchronous programming allows the application to **continue execution without blocking** while waiting for the I/O operation to complete.

### Logic

1. Use the `async` keyword on the method performing the I/O operation.
2. Use the `await` keyword to asynchronously wait for the I/O operation to complete.
3. The method should return `Task` (or `Task<T>` for methods returning a value).
4. The calling code can continue executing other tasks while awaiting the I/O operation.

**Benefits:**

- Improves responsiveness in GUI apps.
- In web apps, frees up threads to handle more requests.

```
using System;
using System.IO;
using System.Threading.Tasks;

class Program
{
    // Async method to read file content
    public static async Task<string> ReadFileAsync(string filePath)
    {
        if (!File.Exists(filePath))
            return "File not found";

        using (StreamReader reader = new StreamReader(filePath))
        {
            string content = await reader.ReadToEndAsync(); // Non-blocking
            return content;
        }
    }
```

```
    // Async method to write to file
    public static async Task WriteFileAsync(string filePath, string content)
    {
        using (StreamWriter writer = new StreamWriter(filePath))
        {
            await writer.WriteLineAsync(content); // Non-blocking
        }
    }
```

```
    static async Task Main()
    {
        string filePath = "test.txt";

        // Write to file asynchronously
        await WriteFileAsync(filePath, "Hello, this is an async file write!");

        // Read file asynchronously
        string result = await ReadFileAsync(filePath);
        Console.WriteLine("File Content:");
        Console.WriteLine(result);

        Console.WriteLine("Async I/O operation completed.");
    }
}
```

### Step-by-Step Explanation

1. **WriteFileAsync:**
   - Uses `await writer.WriteLineAsync()` → writes to the file without blocking the main thread.
2. **ReadFileAsync:**
   - Uses `await reader.ReadToEndAsync()` → reads the file asynchronously.
3. **Main:**
   - Marked as `async Task Main()` to allow `await`.
   - Both write and read operations are **non-blocking**, meaning the thread is free for other work.

## Output

```arduino
File Content:
Hello, this is an async file write!
Async I/O operation completed.
```

### Key Points

- `async` + `await` is the recommended pattern for I/O-bound operations.
- Improves scalability and responsiveness.
- For CPU-bound operations, use `Task.Run` to offload work to a background thread instead.

## Demonstrate Dependency Injection design pattern:

### Problem Statement

- **Dependency Injection** is a design pattern used to **decouple classes** from their dependencies.
- Instead of a class creating its dependencies, they are **provided externally**, typically via constructor, property, or method.
- **Benefits**: Easier testing, maintainability, and flexibility.

### Logic

1. Identify a **service or dependency** that a class needs.
2. Define an **interface** for the dependency (optional but recommended).
3. Inject the dependency via **constructor, property, or method** instead of creating it inside the class.
4. Use a **DI container** (like the built-in container in ASP.NET Core) or manually inject the dependency.

### C# Example (Console App with Constructor Injection)

#### Step 1: Define a Service Interface

csharp Copy code

```csharp
public interface IMessageService
{
    void SendMessage(string message);
}
```

#### Step 2: Implement the Service

csharp Copy code

```csharp
public class EmailService : IMessageService
{
    public void SendMessage(string message)
    {
        Console.WriteLine($"Email sent: {message}");
    }
}
```

### Step 3: Create a Consumer Class

csharp

Copy code

```csharp
public class Notification
{
    private readonly IMessageService _messageService;

    // Constructor Injection
    public Notification(IMessageService messageService)
    {
        _messageService = messageService;
    }

    public void NotifyUser(string message)
    {
        _messageService.SendMessage(message);
    }
}
```

### Step 4: Inject Dependency and Use

csharp

Copy code

```csharp
class Program
{
    static void Main()
    {
        // Manual DI
        IMessageService emailService = new EmailService();
        Notification notification = new Notification(emailService);

        notification.NotifyUser("Welcome to Dependency Injection in C#!");
    }
}
```

### Output:

nginx

```nginx
Email sent: Welcome to Dependency Injection in C#!
```

### Step-by-Step Explanation

1. `Notification` depends on `IMessageService`.
2. Instead of creating `EmailService` inside `Notification`, we **inject it via constructor**.
3. This allows:
   - Replacing `EmailService` with another implementation (e.g., `SmsService`) easily.
   - Easier **unit testing** by injecting a mock service.

### Step 5: Example with Multiple Implementations

```csharp
public class SmsService : IMessageService
{
    public void SendMessage(string message)
    {
        Console.WriteLine($"SMS sent: {message}");
    }
}

// Usage
IMessageService smsService = new SmsService();
Notification notificationSms = new Notification(smsService);
notificationSms.NotifyUser("Hello via SMS!");
```

**Output:**

```yaml
SMS sent: Hello via SMS!
```

### Key Points

- **DI Types:**
  - Constructor Injection (most common)
  - Property Injection
  - Method Injection
- **Benefits:**
  - Loose coupling
  - Easier testing and mocking
  - Flexible and maintainable code

**Design a Product controller showcasing CRUD operations with appropriate HTTP methods.**

## ✅ C# Controller Code: `ProductsController`

csharp

Copy code

```
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;
using System.Linq;

// Model
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public double Price { get; set; }
}
```

Copy code

```
// Controller
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    // In-memory list of products
    private static List<Product> products = new List<Product>
    {
        new Product { Id = 1, Name = "Laptop", Price = 50000 },
        new Product { Id = 2, Name = "Mouse", Price = 500 }
    };

    // GET: api/products
    [HttpGet]
    public ActionResult<IEnumerable<Product>> Get()
    {
        return Ok(products);
    }
```

Copy code

```
    // GET: api/products/{id}
    [HttpGet("{id}")]
    public ActionResult<Product> Get(int id)
    {
        var product = products.FirstOrDefault(p => p.Id == id);
        if (product == null)
            return NotFound();
        return Ok(product);
    }
```

```
    // POST: api/products
    [HttpPost]
    public ActionResult<Product> Post([FromBody] Product newProduct)
    {
        newProduct.Id = products.Max(p => p.Id) + 1;
        products.Add(newProduct);
        return CreatedAtAction(nameof(Get), new { id = newProduct.Id }, newProduct);
    }
```

Copy code

```
    // PUT: api/products/{id}
    [HttpPut("{id}")]
    public IActionResult Put(int id, [FromBody] Product updatedProduct)
    {
        var product = products.FirstOrDefault(p => p.Id == id);
        if (product == null)
            return NotFound();

        product.Name = updatedProduct.Name;
        product.Price = updatedProduct.Price;
        return NoContent();
    }

    // DELETE: api/products/{id}
    [HttpDelete("{id}")]
    public IActionResult Delete(int id)
    {
        var product = products.FirstOrDefault(p => p.Id == id);
        if (product == null)
            return NotFound();

        products.Remove(product);
        return NoContent();
    }
}
```

### 🔎 How It Works

1. **In-memory list** used for demo purposes (`products`).
2. **Routes**:
   - `GET /api/products` → get all products
   - `GET /api/products/{id}` → get product by ID
   - `POST /api/products` → create a new product
   - `PUT /api/products/{id}` → update a product
   - `DELETE /api/products/{id}` → delete a product
3. Uses **ActionResult** and standard HTTP responses like `Ok()`, `CreatedAtAction()`, `NoContent()`, `NotFound()`.

**Demonstrate creating a custom middleware component in ASP.NET Core.**

## ✅ Step 1: Create Custom Middleware Class

csharp

Copy code

```csharp
using Microsoft.AspNetCore.Http;
using System.Threading.Tasks;
using System.Diagnostics;

public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;

    public RequestLoggingMiddleware(RequestDelegate next)
    {
        _next = next;
    }
    public async Task InvokeAsync(HttpContext context)
    {
        // Log request path and method
        var stopwatch = Stopwatch.StartNew();
        Console.WriteLine($"Incoming Request: {context.Request.Method} {context.Request.Path}");

        // Call the next middleware in the pipeline
        await _next(context);

        stopwatch.Stop();
        Console.WriteLine($"Request processed in {stopwatch.ElapsedMilliseconds} ms");
    }
}
```

## ✅ Step 2: Register Middleware in `Program.cs` (or `Startup.cs`)

csharp Copy code

```csharp
using Microsoft.AspNetCore.Builder;
using Microsoft.Extensions.Hosting;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

// Use custom middleware
app.UseMiddleware<RequestLoggingMiddleware>();

app.MapGet("/", () => "Hello World!");
app.MapGet("/products", () => new[] { "Laptop", "Mouse", "Keyboard" });

app.Run();
```

## 🔎 How It Works

1. **Middleware class**
   - Constructor takes `RequestDelegate next` → next middleware in pipeline.
   - `InvokeAsync(HttpContext context)` → logic to execute for each request.
2. **Logging**
   - Logs the HTTP method and request path.
   - Measures request processing time using `Stopwatch`.
3. **Registration**
   - Use `app.UseMiddleware<YourMiddleware>()` to add to pipeline.

## 🔎 Sample Console Output

```
Incoming Request: GET /
Request processed in 5 ms
Incoming Request: GET /products
Request processed in 2 ms
```

## ✅ Notes

- Middleware runs **for every HTTP request**.
- Can perform actions **before and after calling** `_next(context)`.
- Useful for logging, authentication, error handling, performance monitoring, etc.

**Demonstrate unit testing of repository classes using mocks.**

## ✅ Step 1: Create Repository Interface and Service

csharp

```csharp
using System.Collections.Generic;

// Repository interface
public interface IProductRepository
{
    IEnumerable<string> GetAllProducts();
}

// Service that depends on repository
public class ProductService
{
    private readonly IProductRepository _repository;

    public ProductService(IProductRepository repository)
    {
        _repository = repository;
    }

    public int GetProductCount()
    {
        var products = _repository.GetAllProducts();
        return products == null ? 0 : new List<string>(products).Count;
    }
}
```

## ✅ Step 2: Unit Test with Mock Repository

```
using Xunit;
using Moq;
using System.Collections.Generic;

public class ProductServiceTests
{
    [Fact]
    public void GetProductCount_ReturnsCorrectCount()
    {
        // Arrange
        var mockRepo = new Mock<IProductRepository>();
        mockRepo.Setup(r => r.GetAllProducts())
                .Returns(new List<string> { "Laptop", "Mouse", "Keyboard" });

        var service = new ProductService(mockRepo.Object);
        // Act
        int count = service.GetProductCount();
        // Assert
        Assert.Equal(3, count);
    }
```

```
    [Fact]
    public void GetProductCount_ReturnsZero_WhenNoProducts()
    {
        // Arrange
        var mockRepo = new Mock<IProductRepository>();
        mockRepo.Setup(r => r.GetAllProducts())
                .Returns(new List<string>());

        var service = new ProductService(mockRepo.Object);

        // Act
        int count = service.GetProductCount();

        // Assert
        Assert.Equal(0, count);
    }
}
```

### 🔎 How It Works

1. `IProductRepository` → interface to abstract data access.
2. `ProductService` → depends on repository via constructor injection.
3. **Moq** → used to **mock repository behavior** without touching a database.
4. **xUnit** → test framework.
5. `mockRepo.Setup(...)` → defines what the mock should return when a method is called.
6. **Assertions** → validate service behavior using mocked data.

### ✅ Notes

- This approach is **isolated**: you are testing only `ProductService`, not the repository implementation.
- Works well for **unit testing services** in layered applications like ASP.NET Core.
- You can also mock **async methods** using `ReturnsAsync()`.

**Demonstrate the behavior of different dependency injection lifetimes in ASP.NET Core.**

### ✅ C# Code: DI Lifecycle Example

csharp Copy code

```csharp
using System;
using Microsoft.Extensions.DependencyInjection;

// Service interfaces
public interface ITransientService { Guid GetId(); }
public interface IScopedService { Guid GetId(); }
public interface ISingletonService { Guid GetId(); }

// Service implementations
public class TransientService : ITransientService
{
    private readonly Guid _id;
    public TransientService() => _id = Guid.NewGuid();
    public Guid GetId() => _id;
}
```

```csharp
public class ScopedService : IScopedService
{
    private readonly Guid _id;
    public ScopedService() => _id = Guid.NewGuid();
    public Guid GetId() => _id;
}
```

```
public class SingletonService : ISingletonService
{
    private readonly Guid _id;
    public SingletonService() => _id = Guid.NewGuid();
    public Guid GetId() => _id;
}
```

```
class Program
{
    static void Main()
    {
        // Setup DI container
        var serviceCollection = new ServiceCollection();

        // Register services with different lifetimes
        serviceCollection.AddTransient<ITransientService, TransientService>();
        serviceCollection.AddScoped<IScopedService, ScopedService>();
        serviceCollection.AddSingleton<ISingletonService, SingletonService>();

        var serviceProvider = serviceCollection.BuildServiceProvider();

        // Simulate first scope
        Console.WriteLine("=== Scope 1 ===");
        using (var scope1 = serviceProvider.CreateScope())
        {
            var transient1 = scope1.ServiceProvider.GetService<ITransientService>();
            var scoped1 = scope1.ServiceProvider.GetService<IScopedService>();
            var singleton1 = scope1.ServiceProvider.GetService<ISingletonService>();

            var transient2 = scope1.ServiceProvider.GetService<ITransientService>();
            var scoped2 = scope1.ServiceProvider.GetService<IScopedService>();
            var singleton2 = scope1.ServiceProvider.GetService<ISingletonService>();
```

Copy code

```
            Console.WriteLine($"Transient 1: {transient1.GetId()}");
            Console.WriteLine($"Transient 2: {transient2.GetId()}");

            Console.WriteLine($"Scoped 1: {scoped1.GetId()}");
            Console.WriteLine($"Scoped 2: {scoped2.GetId()}");

            Console.WriteLine($"Singleton 1: {singleton1.GetId()}");
            Console.WriteLine($"Singleton 2: {singleton2.GetId()}");
        }
```

```
        // Simulate second scope
        Console.WriteLine("\n=== Scope 2 ===");
        using (var scope2 = serviceProvider.CreateScope())
        {
            var transient = scope2.ServiceProvider.GetService<ITransientService>();
            var scoped = scope2.ServiceProvider.GetService<IScopedService>();
            var singleton = scope2.ServiceProvider.GetService<ISingletonService>();

            Console.WriteLine($"Transient: {transient.GetId()}");
            Console.WriteLine($"Scoped: {scoped.GetId()}");
            Console.WriteLine($"Singleton: {singleton.GetId()}");
        }
    }
}
```

## 🔎 How It Works

| Lifetime | Behavior |
|---|---|
| **Transient** | New instance every time you request the service. |
| **Scoped** | Same instance within a scope, different across scopes. |
| **Singleton** | Same instance across the entire application lifetime. |

- `serviceProvider.CreateScope()` → creates a new **scope** for scoped services.
- Within a scope, **Scoped** instances are reused, but **Transient** is always new.
- **Singleton** instance remains the same across all scopes.

## 🔎 Sample Output

yaml

Copy code

```
=== Scope 1 ===
Transient 1: 3f2c4b0a-1e5d-4a5a-bf21-1f6e9a8f1234
Transient 2: 6e9f4a23-8c56-4b5c-bf45-1a2d8b9c5678
Scoped 1: 1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d
Scoped 2: 1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d
Singleton 1: 9f8e7d6c-5b4a-3c2d-1e0f-123456789abc
Singleton 2: 9f8e7d6c-5b4a-3c2d-1e0f-123456789abc

=== Scope 2 ===
Transient: 7a8b9c0d-1e2f-3a4b-5c6d-8f9e0d1c2b3a
Scoped: 2b3c4d5e-6f7a-8b9c-0d1e-2f3a4b5c6d7e
Singleton: 9f8e7d6c-5b4a-3c2d-1e0f-123456789abc
```

**Demonstrate the steps to configure and use JWT in an ASP.NET Core application.**

## 🔑 Steps to Configure and Use JWT in ASP.NET Core

### 1. Install Required NuGet Packages

Make sure these packages are installed:

bash Copy code

```
Microsoft.AspNetCore.Authentication.JwtBearer
System.IdentityModel.Tokens.Jwt
```

### 2. Configure JWT in `Program.cs`

Add authentication and JWT configuration:

csharp Copy code

```csharp
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

// Add services
builder.Services.AddControllers();

// Configure JWT Authentication
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))
        };
    });

var app = builder.Build();

// Enable Authentication & Authorization
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
app.Run();
```

### 3. Add JWT Settings in `appsettings.json`

```json
"Jwt": {
  "Key": "ThisIsMySuperSecretKey12345",
  "Issuer": "https://yourdomain.com",
  "Audience": "https://yourdomain.com"
}
```

### 4. Create a Token Generator Service

csharp

Copy code

```csharp
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

public class JwtService
{
    private readonly IConfiguration _config;
    public JwtService(IConfiguration config)
    {
        _config = config;
    }

    public string GenerateToken(string username, string role)
    {
        var securityKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]));
        var credentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256);

        var claims = new[]
        {
            new Claim(ClaimTypes.Name, username),
            new Claim(ClaimTypes.Role, role)
        };

        var token = new JwtSecurityToken(
            issuer: _config["Jwt:Issuer"],
            audience: _config["Jwt:Audience"],
            claims: claims,
            expires: DateTime.Now.AddMinutes(30),
            signingCredentials: credentials);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

### 5. Create an Authentication Controller

csharp

Copy code

```csharp
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly JwtService _jwtService;

    public AuthController(JwtService jwtService)
    {
        _jwtService = jwtService;
    }

    [HttpPost("login")]
    public IActionResult Login([FromBody] LoginModel model)
    {
        // Dummy user validation (replace with DB check)
        if (model.Username == "admin" && model.Password == "password")
        {
            var token = _jwtService.GenerateToken(model.Username, "Admin");
            return Ok(new { Token = token });
        }
        return Unauthorized();
    }
}

public class LoginModel
{
    public string Username { get; set; }
    public string Password { get; set; }
}
```

### 6. Secure a Controller with `[Authorize]`

csharp Copy code

```csharp
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class ProductController : ControllerBase
{
    [HttpGet]
    [Authorize]
    public IActionResult GetProducts()
    {
        return Ok(new[] { "Laptop", "Mobile", "Tablet" });
    }
}
```

### 7. Test the Flow

1. Call `POST /api/auth/login` with:

   json Copy code

   ```json
   {
     "username": "admin",
     "password": "password"
   }
   ```

   → You'll get a **JWT token**.

2. Call `GET /api/product` with header:

   makefile Copy code

   ```
   Authorization: Bearer <your-token-here>
   ```

   → You'll get product data only if the token is valid.

**Demonstrate the process of creating and using refresh tokens with JWT in ASP.NET Core.**

## 🔑 Steps to Implement Refresh Tokens

### 1. Extend Your Auth Model

Add fields for both **Access Token** and **Refresh Token**.

csharp Copy code

```csharp
public class AuthResponse
{
    public string AccessToken { get; set; }
    public string RefreshToken { get; set; }
}
```

### 2. Generate a Refresh Token

You can use a cryptographically secure random string as a refresh token.

csharp Copy code

```csharp
using System.Security.Cryptography;

public class TokenService
{
    private readonly IConfiguration _config;
    public TokenService(IConfiguration config)
    {
        _config = config;
    }

    public string GenerateAccessToken(string username, string role)
    {
        var securityKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]));
        var credentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256);

        var claims = new[]
        {
            new Claim(ClaimTypes.Name, username),
            new Claim(ClaimTypes.Role, role)
        };

        var token = new JwtSecurityToken(
            issuer: _config["Jwt:Issuer"],
            audience: _config["Jwt:Audience"],
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(15), // short-lived access token
            signingCredentials: credentials);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
```

```
    public string GenerateRefreshToken()
    {
        var randomNumber = new byte[32];
        using var rng = RandomNumberGenerator.Create();
        rng.GetBytes(randomNumber);
        return Convert.ToBase64String(randomNumber);
    }
}
```

### 3. Store Refresh Tokens

Normally, refresh tokens are stored in a **database** against the user (with expiry). Example:

csharp Copy code

```
public class UserToken
{
    public string Username { get; set; }
    public string RefreshToken { get; set; }
    public DateTime ExpiryDate { get; set; }
}
```

### 4. Update Login to Return Both Tokens

csharp Copy code

```
[HttpPost("login")]
public IActionResult Login([FromBody] LoginModel model)
{
    if (model.Username == "admin" && model.Password == "password")
    {
        var accessToken = _tokenService.GenerateAccessToken(model.Username, "Admin");
        var refreshToken = _tokenService.GenerateRefreshToken();

        // store refresh token in DB (or in-memory for demo)
        _userTokens[model.Username] = new UserToken
        {
            Username = model.Username,
            RefreshToken = refreshToken,
            ExpiryDate = DateTime.UtcNow.AddDays(7) // refresh token valid for 7 days
        };

        return Ok(new AuthResponse
        {
            AccessToken = accessToken,
            RefreshToken = refreshToken
        });
    }

    return Unauthorized();
}
```

## 5. Create a Refresh Endpoint

csharp Copy code

```csharp
[HttpPost("refresh")]
public IActionResult Refresh([FromBody] AuthResponse tokenModel)
{
    if (tokenModel is null) return BadRequest("Invalid request");

    var userToken = _userTokens.Values
        .FirstOrDefault(u => u.RefreshToken == tokenModel.RefreshToken);

    if (userToken == null || userToken.ExpiryDate <= DateTime.UtcNow)
        return Unauthorized("Invalid or expired refresh token");

    // generate new tokens
    var newAccessToken = _tokenService.GenerateAccessToken(userToken.Username, "Admin");
    var newRefreshToken = _tokenService.GenerateRefreshToken();

    // update refresh token in store
    userToken.RefreshToken = newRefreshToken;
    userToken.ExpiryDate = DateTime.UtcNow.AddDays(7);

    return Ok(new AuthResponse
    {
        AccessToken = newAccessToken,
        RefreshToken = newRefreshToken
    });
}
```

## 🔄 Workflow

1. User logs in → receives **Access Token (short-lived) + Refresh Token (long-lived)**.
2. Access Token expires after 15 mins.
3. Client sends **Refresh Token** to `/api/auth/refresh`.
4. Server validates refresh token → issues **new Access Token + new Refresh Token**.
5. User stays logged in without re-entering credentials.

**Demonstrate using Azure Key Vault to securely store and retrieve secrets for JWT token generation.**

## 🔑 Why Use Azure Key Vault?

Instead of keeping the JWT signing key in `appsettings.json`, we move it to **Azure Key Vault** for improved security. The app retrieves it at runtime.

## ⚙️ Steps

### 1. Create a Secret in Azure Key Vault

- Go to **Azure Portal → Key Vaults → Your Vault → Secrets**
- Create a new secret, e.g.:
  - **Name:** `JwtSecret`
  - **Value:** `ThisIsMySuperSecretKey12345`

### 2. Add Required NuGet Packages

```bash
dotnet add package Azure.Identity
dotnet add package Azure.Security.KeyVault.Secrets
```

### 3. Configure Program.cs to Use Azure Key Vault

```csharp
using Azure.Identity;

var builder = WebApplication.CreateBuilder(args);

// Load secrets from Azure Key Vault
var keyVaultUrl = builder.Configuration["KeyVaultConfig:VaultUrl"];
builder.Configuration.AddAzureKeyVault(new Uri(keyVaultUrl), new DefaultAzureCredential());

// Add services
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// JWT Authentication setup
builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer("Bearer", options =>
    {
        var jwtKey = builder.Configuration["JwtSecret"]; // Fetched from Key Vault
        var keyBytes = System.Text.Encoding.UTF8.GetBytes(jwtKey);

        options.TokenValidationParameters = new Microsoft.IdentityModel.Tokens.TokenValidationParameters
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new Microsoft.IdentityModel.Tokens.SymmetricSecurityKey(keyBytes)
        };
    });

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
app.Run();
```

### 4. Update `appsettings.json`

```json
"KeyVaultConfig": {
  "VaultUrl": "https://your-keyvault-name.vault.azure.net/"
}
```

### 5. Generate JWT Tokens Using Secret from Key Vault

csharp

Copy code

```csharp
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

public class JwtService
{
    private readonly IConfiguration _config;
    public JwtService(IConfiguration config)
    {
        _config = config;
    }

    public string GenerateToken(string username, string role)
    {
        var key = _config["JwtSecret"]; // Loaded from Key Vault
        var securityKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(key));
        var credentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256);

        var claims = new[]
        {
            new Claim(ClaimTypes.Name, username),
            new Claim(ClaimTypes.Role, role)
        };

        var token = new JwtSecurityToken(
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(30),
            signingCredentials: credentials);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

### 6. Usage

- When the app starts, it connects to **Azure Key Vault** using **Managed Identity** or **Azure credentials**.
- It fetches the **JWT secret key** from Key Vault.
- JWT tokens are generated and validated using this secret.

**Demonstrate using JWT authentication with role-based and policy-based authorization in ASP.NET Core:**

```
// Add Authorization with Roles & Policies
builder.Services.AddAuthorization(options =>
{
    // Policy based on Claim
    options.AddPolicy("RequireAdmin", policy => policy.RequireRole("Admin"));
    options.AddPolicy("MustBeHR", policy => policy.RequireClaim("Department", "HR"));
});
```

### 5. Authentication Controller

csharp

```
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly JwtService _jwtService;

    public AuthController(JwtService jwtService)
    {
        _jwtService = jwtService;
    }

    [HttpPost("login")]
    public IActionResult Login([FromBody] LoginModel model)
    {
        // Dummy validation
        if (model.Username == "alice" && model.Password == "password")
        {
            var token = _jwtService.GenerateToken("alice", "Admin", "HR");
            return Ok(new { Token = token });
        }

        if (model.Username == "bob" && model.Password == "password")
        {
            var token = _jwtService.GenerateToken("bob", "User", "Finance");
            return Ok(new { Token = token });
        }

        return Unauthorized();
    }
}
```

```csharp
public class LoginModel
{
    public string Username { get; set; }
    public string Password { get; set; }
}
```

### 6. Role-based and Policy-based Protected Controller

csharp

```csharp
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class ProductController : ControllerBase
{
    // Role-based Authorization
    [HttpGet("admin")]
    [Authorize(Roles = "Admin")]
    public IActionResult GetForAdmin()
    {
        return Ok("This endpoint is accessible only by Admin role.");
    }

    // Policy-based Authorization
    [HttpGet("hr")]
    [Authorize(Policy = "MustBeHR")]
    public IActionResult GetForHR()
    {
        return Ok("This endpoint is accessible only by HR department users.");
    }

    // Open to all authenticated users
    [HttpGet("all")]
    [Authorize]
    public IActionResult GetForAll()
    {
        return Ok("This endpoint is accessible by any authenticated user.");
    }
}
```

## 🔄 Test the Flow

1. **Login as Alice →** `POST /api/auth/login`

```json
{ "username": "alice", "password": "password" }
```

→ Alice gets **Role = Admin**, **Department = HR**.

✅ She can call `/api/product/admin` and `/api/product/hr`.

2. **Login as Bob →** `POST /api/auth/login`

```json
{ "username": "bob", "password": "password" }
```

→ Bob gets **Role = User**, **Department = Finance**.

❌ He cannot call `/api/product/admin` or `/api/product/hr`.
✅ But he can call `/api/product/all`.

**Demonstrate creating custom middleware for centralized exception handling in ASP.NET Core:**

👉 **This approach ensures all exceptions are handled in one place instead of scattered try-catch blocks in controllers.**

## 🔧 Steps to Implement

### 1. Create Exception Handling Middleware

csharp Copy code

```csharp
using Microsoft.AspNetCore.Http;
using System.Net;
using System.Text.Json;

public class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionHandlingMiddleware> _logger;

    public ExceptionHandlingMiddleware(RequestDelegate next, ILogger<ExceptionHandlingMiddleware>
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            // Call the next middleware in pipeline
            await _next(context);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "An unhandled exception occurred.");

            await HandleExceptionAsync(context, ex);
        }
    }
```

```
    private static Task HandleExceptionAsync(HttpContext context, Exception exception)
    {
        context.Response.ContentType = "application/json";

        // Map different exception types to different status codes
        var statusCode = exception switch
        {
            ArgumentNullException => (int)HttpStatusCode.BadRequest,
            UnauthorizedAccessException => (int)HttpStatusCode.Unauthorized,
            _ => (int)HttpStatusCode.InternalServerError
        };

        context.Response.StatusCode = statusCode;

        var errorResponse = new
        {
            StatusCode = statusCode,
            Message = exception.Message,
            Details = exception.InnerException?.Message
        };

        var json = JsonSerializer.Serialize(errorResponse);
        return context.Response.WriteAsync(json);
    }
}
```

### 2. Create an Extension Method for Cleaner Registration

csharp

```
public static class ExceptionHandlingMiddlewareExtensions
{
    public static IApplicationBuilder UseGlobalExceptionHandling(this IApplicationBuilder builder)
    {
        return builder.UseMiddleware<ExceptionHandlingMiddleware>();
    }
}
```

### 3. Register Middleware in `Program.cs`

csharp

```csharp
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddLogging();

var app = builder.Build();

// Register our custom middleware
app.UseGlobalExceptionHandling();

app.MapControllers();
app.Run();
```

### 4. Example Controller to Test

csharp

```csharp
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class TestController : ControllerBase
{
    [HttpGet("error")]
    public IActionResult ThrowError()
    {
        throw new Exception("Something went wrong!");
    }

    [HttpGet("badrequest")]
    public IActionResult ThrowBadRequest()
    {
        throw new ArgumentNullException("id", "Id parameter is missing!");
    }
}
```

## ✅ Output Examples

- Call `/api/test/error` → returns:

```json
{
  "statusCode": 500,
  "message": "Something went wrong!",
  "details": null
}
```

- Call `/api/test/badrequest` → returns:

```json
{
  "statusCode": 400,
  "message": "Id parameter is missing!",
  "details": null
}
```

**Demonstrate writing a custom filter attribute that measures and logs the duration of controller action execution:**

## 🔧 Steps

### 1. Create the Custom Filter Attribute

csharp Copy code

```csharp
using Microsoft.AspNetCore.Mvc.Filters;
using Microsoft.Extensions.Logging;
using System.Diagnostics;

public class ExecutionTimeFilterAttribute : ActionFilterAttribute
{
    private readonly ILogger<ExecutionTimeFilterAttribute> _logger;
    private Stopwatch _stopwatch;

    public ExecutionTimeFilterAttribute(ILogger<ExecutionTimeFilterAttribute> logger)
    {
        _logger = logger;
    }

    public override void OnActionExecuting(ActionExecutingContext context)
    {
        _stopwatch = Stopwatch.StartNew();
    }
```

```csharp
    public override void OnActionExecuted(ActionExecutedContext context)
    {
        _stopwatch.Stop();
        var elapsed = _stopwatch.ElapsedMilliseconds;

        var controllerName = context.ActionDescriptor.RouteValues["controller"];
        var actionName = context.ActionDescriptor.RouteValues["action"];

        _logger.LogInformation(
            "Execution time for {Controller}/{Action}: {Elapsed} ms",
            controllerName,
            actionName,
            elapsed
        );
    }
}
```

### 2. Apply the Filter Globally (Optional)

Register it in `Program.cs` if you want it applied to all controllers:

```csharp
builder.Services.AddControllers(options =>
{
    options.Filters.Add<ExecutionTimeFilterAttribute>();
});
```

> This way, every controller action will be logged automatically.

### 3. Or Apply It to Specific Controllers/Actions

Copy code

```csharp
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class ProductController : ControllerBase
{
    [HttpGet("all")]
    [ServiceFilter(typeof(ExecutionTimeFilterAttribute))] // Apply filter here
    public IActionResult GetProducts()
    {
        // Simulate processing time
        Thread.Sleep(500);
        return Ok(new[] { "Laptop", "Mobile", "Tablet" });
    }
}
```

### 4. Register the Filter in Dependency Injection

If using `[ServiceFilter]`, register it in `Program.cs`:

```csharp
builder.Services.AddScoped<ExecutionTimeFilterAttribute>();
```

## ✅ Example Log Output

When calling `/api/product/all`, you'll see something like this in logs:

```yaml
info: ExecutionTimeFilterAttribute[0]
      Execution time for Product/GetProducts: 502 ms
```

**Demonstrate uploading and downloading files in an ASP.NET Core API.**

## 🔧 Steps

### 1. Configure File Storage Path

For simplicity, let's store uploaded files in a folder called `Uploads`.

```csharp
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

var app = builder.Build();

app.UseStaticFiles(); // optional, to serve files directly if needed

app.MapControllers();
app.Run();
```

### 2. File Upload Endpoint

```csharp
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class FileController : ControllerBase
{
    private readonly string _storagePath = Path.Combine(Directory.GetCurrentDirectory(), "Uploads");

    public FileController()
    {
        if (!Directory.Exists(_storagePath))
        {
            Directory.CreateDirectory(_storagePath);
        }
    }
```

```
    // Upload Single File
    [HttpPost("upload")]
    public async Task<IActionResult> UploadFile(IFormFile file)
    {
        if (file == null || file.Length == 0)
            return BadRequest("No file uploaded.");

        var filePath = Path.Combine(_storagePath, file.FileName);

        using (var stream = new FileStream(filePath, FileMode.Create))
        {
            await file.CopyToAsync(stream);
        }

        return Ok(new { FileName = file.FileName, FilePath = filePath });
    }
```

```
// Upload Multiple Files
[HttpPost("upload-multiple")]
public async Task<IActionResult> UploadMultiple(List<IFormFile> files)
{
    if (files == null || files.Count == 0)
        return BadRequest("No files uploaded.");

    var uploadedFiles = new List<string>();

    foreach (var file in files)
    {
        var filePath = Path.Combine(_storagePath, file.FileName);

        using (var stream = new FileStream(filePath, FileMode.Create))
        {
            await file.CopyToAsync(stream);
        }
        uploadedFiles.Add(file.FileName);
    }

    return Ok(new { UploadedFiles = uploadedFiles });
}
```

### 3. File Download Endpoint

csharp

```
// Download File
[HttpGet("download/{fileName}")]
public async Task<IActionResult> DownloadFile(string fileName)
{
    var filePath = Path.Combine(_storagePath, fileName);

    if (!System.IO.File.Exists(filePath))
        return NotFound("File not found.");

    var memory = new MemoryStream();
    using (var stream = new FileStream(filePath, FileMode.Open))
    {
        await stream.CopyToAsync(memory);
    }
    memory.Position = 0;

    var contentType = "application/octet-stream";
    return File(memory, contentType, fileName);
}
```

### 4. Test with Postman

- **Upload Single File:**
  - `POST /api/file/upload`
  - Body → `form-data` → key= `file`, type= `File`
- **Upload Multiple Files:**
  - `POST /api/file/upload-multiple`
  - Body → `form-data` → multiple keys= `files`
- **Download File:**
  - `GET /api/file/download/{filename}`

## ✅ Example

1. Upload `report.pdf` → stored in `Uploads/report.pdf`
2. Download via `GET /api/file/download/report.pdf` → file returned

**Demonstrate implementing Repository and Unit of Work in an EF Core project.**

## Step 1: Define the EF Core DbContext

csharp Copy code

```csharp
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }
}
```

### Sample Entities

csharp Copy code

```csharp
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    public int CategoryId { get; set; }
    public Category Category { get; set; }
}
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; }
    public ICollection<Product> Products { get; set; }
}
```

## Step 2: Create a Generic Repository

csharp Copy code

```csharp
public interface IRepository<T> where T : class
{
    Task<IEnumerable<T>> GetAllAsync();
    Task<T> GetByIdAsync(int id);
    Task AddAsync(T entity);
    void Update(T entity);
    void Delete(T entity);
}
```

## Implementation

csharp

Copy code

```csharp
using Microsoft.EntityFrameworkCore;

public class GenericRepository<T> : IRepository<T> where T : class
{
    protected readonly AppDbContext _context;
    private readonly DbSet<T> _dbSet;

    public GenericRepository(AppDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }

    public async Task<IEnumerable<T>> GetAllAsync()
    {
        return await _dbSet.ToListAsync();
    }

    public async Task<T> GetByIdAsync(int id)
    {
        return await _dbSet.FindAsync(id);
    }

    public async Task AddAsync(T entity)
    {
        await _dbSet.AddAsync(entity);
    }

    public void Update(T entity)
    {
        _dbSet.Update(entity);
    }

    public void Delete(T entity)
    {
        _dbSet.Remove(entity);
    }
}
```

## 🏗️ Step 3: Unit of Work Pattern

csharp

```csharp
public interface IUnitOfWork : IDisposable
{
    IRepository<Product> Products { get; }
    IRepository<Category> Categories { get; }
    Task<int> SaveAsync();
}
```

### Implementation

csharp

Copy code

```csharp
public class UnitOfWork : IUnitOfWork
{
    private readonly AppDbContext _context;

    public IRepository<Product> Products { get; }
    public IRepository<Category> Categories { get; }

    public UnitOfWork(AppDbContext context)
    {
        _context = context;
        Products = new GenericRepository<Product>(_context);
        Categories = new GenericRepository<Category>(_context);
    }

    public async Task<int> SaveAsync()
    {
        return await _context.SaveChangesAsync();
    }

    public void Dispose()
    {
        _context.Dispose();
    }
}
```

## 🧱 Step 4: Register Services in `Program.cs`

csharp

Copy code

```csharp
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();

builder.Services.AddControllers();

var app = builder.Build();

app.MapControllers();
app.Run();
```

## 🧱 Step 5: Use in Controller

csharp

Copy code

```csharp
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly IUnitOfWork _unitOfWork;

    public ProductsController(IUnitOfWork unitOfWork)
    {
        _unitOfWork = unitOfWork;
    }

    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var products = await _unitOfWork.Products.GetAllAsync();
        return Ok(products);
    }
```

```
    [HttpPost]
    public async Task<IActionResult> Add(Product product)
    {
        await _unitOfWork.Products.AddAsync(product);
        await _unitOfWork.SaveAsync();
        return Ok(product);
    }

    [HttpDelete("{id}")]
    public async Task<IActionResult> Delete(int id)
    {
        var product = await _unitOfWork.Products.GetByIdAsync(id);
        if (product == null) return NotFound();

        _unitOfWork.Products.Delete(product);
        await _unitOfWork.SaveAsync();
        return NoContent();
    }
}
```

## ✅ Benefits

- **Repository Pattern** → Encapsulates data access logic.
- **Unit of Work Pattern** → Coordinates changes across repositories and ensures atomic commits.
- **Testability** → Easier mocking for unit tests.